

Projet C - Séquence 1.2 - Gestion des utilisateurs (backend)

Création des utilisateurs :

auteur@test.com
auteur2@test.com
lecteur@test.com

Modification des roles :

```
-- Pour author@test.com
UPDATE auth.users SET raw_app_meta_data = raw_app_meta_data || '{"role":
"author"}'::jsonb WHERE email = 'author@test.com';
```

```
UPDATE auth.users SET raw_app_meta_data = raw_app_meta_data || '{"role":
"author"}'::jsonb WHERE email = 'author2@test.com';
```

```
-- Pour reader@test.com
UPDATE auth.users SET raw_app_meta_data = raw_app_meta_data || '{"role":
"reader"}'::jsonb WHERE email = 'lecteur@test.com';
```

Création des RLS policy

Pour un forum communautaire, il est essentiel de sécuriser l'accès aux données en utilisant les **Row-Level Security (RLS)** de Supabase. Voici les règles RLS que je recommande pour chaque table, en tenant compte des besoins typiques d'un forum :

1. Table **Utilisateur**

- **Lecture** : Un utilisateur ne peut voir que son propre profil.
 - **Insertion** : Autoriser la création de nouveaux utilisateurs (inscription).
 - **Mise à jour** : Un utilisateur ne peut modifier que son propre profil.
 - **Suppression** : Un utilisateur ne peut supprimer que son propre profil.
-

2. Table **Sujet**

- **Lecture** : Tous les utilisateurs peuvent voir tous les sujets.
 - **Insertion** : Un utilisateur peut créer un sujet (lié à son `auteur_id`).
 - **Mise à jour** : Un utilisateur ne peut modifier que ses propres sujets.
 - **Suppression** : Un utilisateur ne peut supprimer que ses propres sujets
-

3. Table **Commentaire**

- **Lecture** : Tous les utilisateurs peuvent voir tous les commentaires.
 - **Insertion** : Un utilisateur peut ajouter un commentaire (lié à son `auteur_id` et à un `sujet_id` existant).
 - **Mise à jour** : Un utilisateur ne peut modifier que ses propres commentaires.
 - **Suppression** : Un utilisateur ne peut supprimer que ses propres commentaires
-

4. Règles supplémentaires

- **Vérification de l'authentification** : Toutes les règles supposent que l'utilisateur est authentifié (`auth.uid()`).
-

Souhaitez-vous que je génère le script SQL complet pour ces règles RLS ? Si oui, précisez si vous voulez :

- Un rôle `admin` avec des droits étendus.
- Des restrictions supplémentaires (ex : limiter la suppression aux administrateurs uniquement).

Script SQL pour le RLS

```
-- Activation du RLS pour toutes les tables
ALTER TABLE Utilisateur ENABLE ROW LEVEL SECURITY;
ALTER TABLE Sujet ENABLE ROW LEVEL SECURITY;
ALTER TABLE Commentaire ENABLE ROW LEVEL SECURITY;

-- =====
-- Règles pour la table Utilisateur
-- =====

-- Lecture : Un utilisateur ne peut voir que son propre profil
CREATE POLICY "Utilisateur: Lecture propre profil"
ON Utilisateur
FOR SELECT
USING (auth.uid() = id::text);
```

```

-- Insertion : Autoriser la création de nouveaux utilisateurs (inscription)
CREATE POLICY "Utilisateur: Insertion"
ON Utilisateur
FOR INSERT
WITH CHECK (true);

-- Mise à jour : Un utilisateur ne peut modifier que son propre profil
CREATE POLICY "Utilisateur: Mise à jour propre profil"
ON Utilisateur
FOR UPDATE
USING (auth.uid() = id::text);

-- Suppression : Un utilisateur ne peut supprimer que son propre profil
CREATE POLICY "Utilisateur: Suppression propre profil"
ON Utilisateur
FOR DELETE
USING (auth.uid() = id::text);

-- =====
-- Règles pour la table Sujet
-- =====

-- Lecture : Tous les utilisateurs peuvent voir tous les sujets
CREATE POLICY "Sujet: Lecture tous"
ON Sujet
FOR SELECT
USING (true);

-- Insertion : Un utilisateur peut créer un sujet (lié à son auteur_id)
CREATE POLICY "Sujet: Insertion"
ON Sujet
FOR INSERT
WITH CHECK (auth.uid() = auteur_id::text);

-- Mise à jour : Un utilisateur ne peut modifier que ses propres sujets
CREATE POLICY "Sujet: Mise à jour propres sujets"
ON Sujet
FOR UPDATE
USING (auth.uid() = auteur_id::text);

-- Suppression : Un utilisateur ne peut supprimer que ses propres sujets
CREATE POLICY "Sujet: Suppression propres sujets"
ON Sujet
FOR DELETE
USING (auth.uid() = auteur_id::text);

-- =====
-- Règles pour la table Commentaire
-- =====

-- Lecture : Tous les utilisateurs peuvent voir tous les commentaires
CREATE POLICY "Commentaire: Lecture tous"
ON Commentaire
FOR SELECT
USING (true);

-- Insertion : Un utilisateur peut ajouter un commentaire (lié à son auteur_id et à un sujet_id existant)
CREATE POLICY "Commentaire: Insertion"
ON Commentaire
FOR INSERT
WITH CHECK (
    auth.uid() = auteur_id::text
    AND EXISTS (SELECT 1 FROM Sujet WHERE id = sujet_id)
);

-- Mise à jour : Un utilisateur ne peut modifier que ses propres commentaires
CREATE POLICY "Commentaire: Mise à jour propres commentaires"
ON Commentaire
FOR UPDATE
USING (auth.uid() = auteur_id::text);

-- Suppression : Un utilisateur ne peut supprimer que ses propres commentaires
CREATE POLICY "Commentaire: Suppression propres commentaires"

```

```
ON Commentaire  
FOR DELETE  
USING (auth.uid() = auteur_id::text);
```